

SmartRPD — 2D Design (jaw-struct L2) Load & Save

Decoding the desktop jaw-struct format, wiring the web load/display path, and producing a desktop-compatible `.txt` on Save.

Date: 8 June 2026 · Endpoint: `POST /api/smartRPD/jawstruct/l2/getall` · Prepared for: nyuntsinh96@gmail.com

1. Objective

Make the web app understand and reproduce the desktop's per-case 2D design format ("jaw-struct L2"). Two deliverables:

- Load / display** — decode the backend's base64 jaw-struct text and render the design (tooth presence, rests, clasps, bars, mesh, major connector) on the 2D arch.
- Save** — export the current design as a desktop-format `.txt` file (replacing the old JSON download) so it can be compared with / consumed by the desktop client.

2. Reverse-engineered the format

The integer codes in the jaw-struct text come from Unity DLL enums. Using the reference `StructData.cs` and two real samples (`03_jawstruct_upper/lower_jaw.txt`), every code was mapped to a web component id and **validated against the actual data**.

Field (enum)	Code → web component	Status
Posterior Rest Type	1 → onlay rest · 2 → rest-seat	2 confirmed
Retainer Type	1 → retainer-clasp · 2 → ring-clasp · 3 → bar	1,3 confirmed
Retainer Bar Category (shape)	0→I 1→S 2→U 3→Y 4→T bar	I confirmed
Mesh Type (spans)	1→hole 2→tori 3→stripe 4→cross 5→flange	hole confirmed
Major Connector Type	2→horseshoe (upper) · 7→lingual-plate (lower) · ...	2,7 confirmed

Key findings (caught only because we checked real data)

- BUG** **Tooth presence was inverted in the old code.** The C# enum is `present = 0, missing = 1` — the opposite of what the web codec assumed. Left unfixed, every present tooth would have shown as missing (and components dropped off the real abutments). **Fixed.**
- The "Reciprocating · Tooth Type" field is noise** — it is `2` on every present tooth and `0` on every missing tooth in both samples (correlated with presence, not a real per-tooth choice). Mapping it would have dropped a plate onto every present tooth, so it is deliberately ignored.
 - The retainer is composite** — clasp/ring/bar comes from *Retainer Type*, but a bar's shape (I/S/U/Y/T) comes from a second field, *Retainer Bar Category*. The bar's mesial/distal side lives in *Retainer Bar Type* (which the previous comments had swapped with Category).
 - Mesh and major connector are not per-tooth** — mesh is stored as spans over missing teeth; the major connector is a single jaw-level value.

3. Load / display implementation

Built a clean decode → resolve → apply pipeline. The codec stays pure (no DOM) so it is unit-testable; the placement step reuses the app's real placement helpers so a loaded design behaves exactly like a hand-placed one.

File	Role
<code>jawStructCodes.js</code>	Filled the enum → component maps; removed the noise field; fixed swapped comments.
<code>jawStructCodec.js</code>	Fixed the presence inversion; added pure <code>resolveJawStructDesign()</code> (retainer composite, mesh-span expansion, major-connector resolution).

jawStructApply.js new	Applies the design to the arch in the correct order: presence → mesh → rests/clasps/bars → major connector.
2DAnnotation.js	Wired <code>fetchJawStruct()</code> to decode → resolve → apply → render.

Note: the automatic fetch-on-open is **still paused** (per your instruction — the 2D state currently restores from localStorage). All the decode/apply code is ready; flipping one line enables it.

4. Verification

A throwaway Node harness ran the **real** codec over both sample files. **All assertions passed:** the present/missing split, the abutment components (upper 17/27 rest+clasp, 15 rest, 23 clasp; lower 37 rest+clasp, **45 rest + I-bar**), the mesh-hole spans (24–26, 46–47) and the major connectors (horseshoe / lingual-plate) all decoded correctly.

5. Save button: JSON → desktop .txt

The Save button now downloads the jaw-struct text (one file per jaw: `case_<id>_jawstruct_upper_jaw.txt` / `..._lower_jaw.txt`). localStorage persistence still uses JSON, so nothing about restore changed.

6. Complete encoder

The encoder was rewritten to emit a **complete 720-line desktop-format file** derived from the live design — all 26 fields per tooth plus the full tail (Mesh, Major Connector, the 16×16 Minor-Connector grid, Ball Connectors). Component values (presence, condition, rests, retainer/bar, mesh spans, major connector) are computed from the placed components; fields the web does not model use the desktop's default values.

Round-trip accuracy vs the desktop samples

Difference	Lines	Reason
Timestamp header	1	Stamps the current time — inherent.
Tooth Position X/Y/Z	upper 11 · lower 4	The originals contain uninitialised memory garbage (e.g. <code>6.6e-316</code>); we write a clean <code>0</code> .
Pr Config (rest position)	upper 2	The web doesn't track a rest's mesial/distal position; we default to mesial.
Minor-Connector path edits	upper 5 · lower 3	The web doesn't model the 16×16 path-activation grid; we default to the desktop's initial "all on".

Everything the web actually models round-trips faithfully. The remaining differences are the timestamp, the original's garbage bytes, and two structures the web has no concept of.

7. Status & what's intentionally not done

- paused** Automatic load-on-open (the `fetchJawStruct()` call) is left commented per your decision.
- limitation** A web-designed case only fills the file once components are actually placed in the app; an empty/localStorage-only state encodes a near-empty file (this is why the first `case_2001` export looked sparse).
- limitation** Per-rest position and the minor-connector grid are not modelled by the web, so those specific lines won't byte-match a desktop export.

8. Files changed

File	Change
src/js/2D/jawStructCodes.js	Enum → component maps filled & trimmed.
src/js/2D/jawStructCodec.js	Presence fix; pure resolver; complete encoder.

src/js/2D/jawStructApply.js	New — applies a decoded design to the arch.
src/js/2D/2DAnnotation.js	Pipeline wiring; added a text-file download helper.
src/js/2D/annotationLocks.js	Save button now downloads <code>.txt</code> instead of JSON.